

Eksperimenti jakog i slabog skaliranja -Python & Rust-

Hardverske karakteristike

1. Procesor: Intel(R) Core(TM) i7-6700HQ CPU @ 2.60GHz
2. Radni takt: 2,60 GHz
3. Broj jezgara: 4 fizička jezgra / 8 logičkih procesora
4. Cache memorija:
 - a. L1: 256 KB
 - b. L2: 1,0 MB
 - c. L3: 6,0 MB
5. RAM: 16,0 GB DDR4
6. Operativni sistem: Microsoft Windows 10 Pro

Softverske karakteristike

- numpy 1.26.4
- pandas 2.2.2
- python-dateutil 2.9.0.post0
- six 1.17.0
- tzdata 2025.3
- Python 3.10.7
- Rust 1.93.1
- Cargo 1.93.1

Metodologija

Unutar *python/report/* se nalaze skripte za jako i slabo skaliranje pajton и паст implementacija rešenja.

- Izvršeno je po 30 pokretanja za svaku konfiguraciju
- Meri se vreme od učitavanja podataka do upisivanja u csv
- Rezultati se pamte i formira se konačni rezultat koji sadrži srednju vrednost, standardnu devijaciju i broj autlajera
- Konačni rezultati se beleže unutar csv fajla

Jako skaliranje (Amdalov zakon)

Teorija

Amdalov zakon predviđa ubrzanje kod fiksne veličine problema.

Formula: **Speedup** = $1 / (s + p / N)$

Parametri:

- sekvencijalni deo S
- paralelni deo $P = 1 - S$
- broj procesora N

Definiše maksimum koji možemo dobiti paralelizacijom. Maksimum je ograničen sekvencijalnim delom koda tj. onim delom koji ne može da se paralelizuje.

Eksperiment

U konkretnom eksperimentu, rađeno je merenje za 100 mašina, 10000 simulacija po mašini pri čemu svaka simulacija vrši računanja za narednih 30 godina.

Podaci za Python:

Broj jezgara	Broj simulacija	Mean Time (s)	Std Dev	Outliers (IQR)
1	1,000,000	68.2480	1.4737	0
2	1,000,000	39.8269	1.2639	0
3	1,000,000	31.8663	4.359	3
4	1,000,000	29.8306	1.1646	4
5	1,000,000	26.6395	1.4009	0
6	1,000,000	25.4830	1.1031	0
7	1,000,000	23.3759	1.5475	0
8	1,000,000	23.0581	4.428	6

Podaci za Rust:

Broj jezgara	Broj simulacija	Mean Time (s)	Std Dev	Outliers (IQR)
1	1,000,000	3.2053	0.1316	1
2	1,000,000	1.7114	0.0509	4
3	1,000,000	1.2548	0.0761	0
4	1,000,000	1.0443	0.0456	1
5	1,000,000	0.8.71	0.0427	1
6	1,000,000	0.8220	0.0771	2
7	1,000,000	0.7381	0.0505	1
8	1,000,000	0.7269	0.0540	0

Ubrzanje

Ubrzanje se računa kao odnos vremena za koje se kod izvrši na jednom jezgri i na p procesora.

$$S(p) = T(1) / T(p)$$

U mom eksperimentu za Python implementaciju, T1 je 68.2480s a T(8) je 23.0581s. Ubrzanje S(8) je približno jednako 2.96.

Za Rust implementaciju, T1 je 3.2053s a T(8) je 0.7269s. Ubrzanje S(8) je približno jednako 4.41.

Efikasnost

Efikasnost računamo po formuli $E(p) = S(p) / p$ gde je p broj procesora. U mom slučaju, imam 8 procesora tako da je efikasnost za Python 37% dok je za Rust 55.1%.

Procena serijskog i paralelnog dela

Za procenu serijskog i paralelnog dela koristićemo Amdalov zakon i konkretne rezultate izmerene u eksperimentu.

$$\text{Speedup} = 1 / (s + p / N)$$

- N je 8
- p je 1 – s
- speedup S(8) je 2.96

Rešavamo po s i dobijamo da je s približno jednako 0.243 što znači da je za Python implementaciju procenat serijskog dela oko 24.3% a paralelnog dela 75.7%.

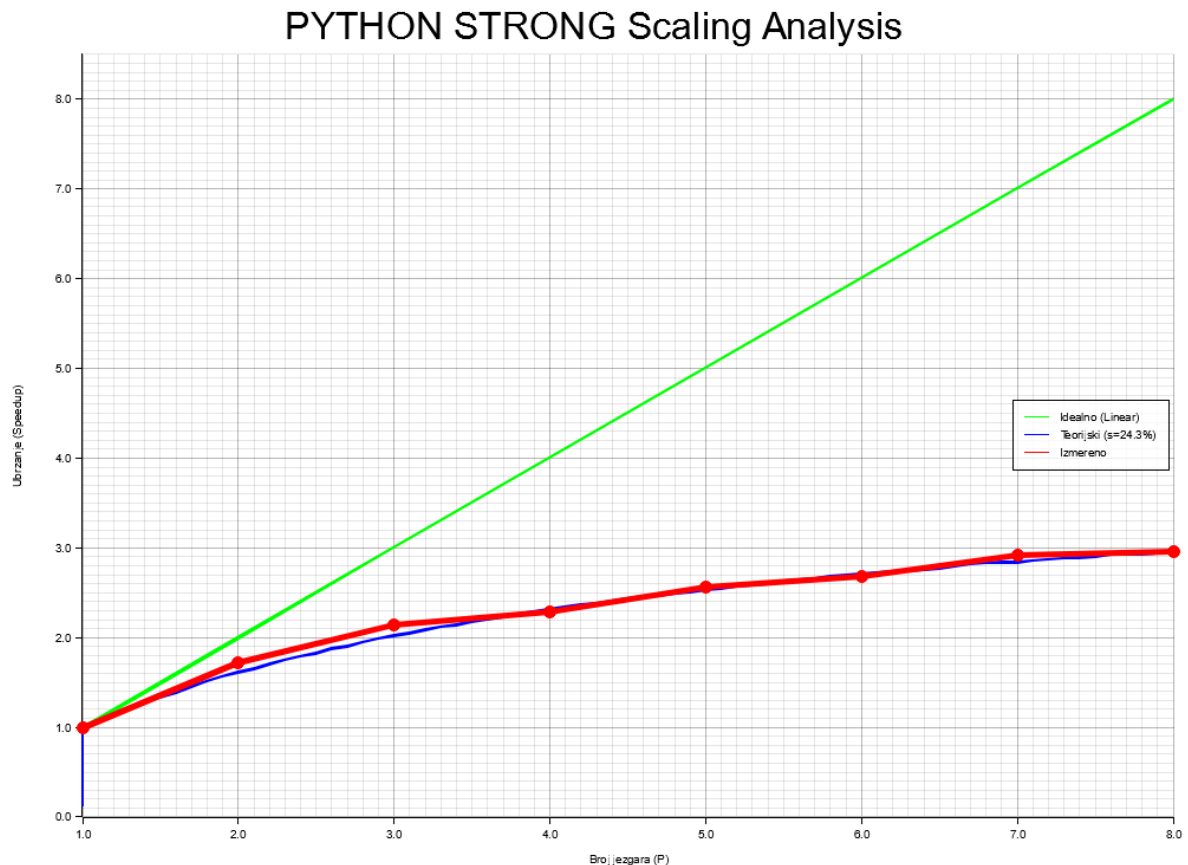
Što se tiče Rust implementacije, parametri su bolji. Procenat serijskog dela koda je približno 11.6% dok je procenat dela koda koji se može paralelizovati 88.4%

Maksimalno teorijskog ubrzanje

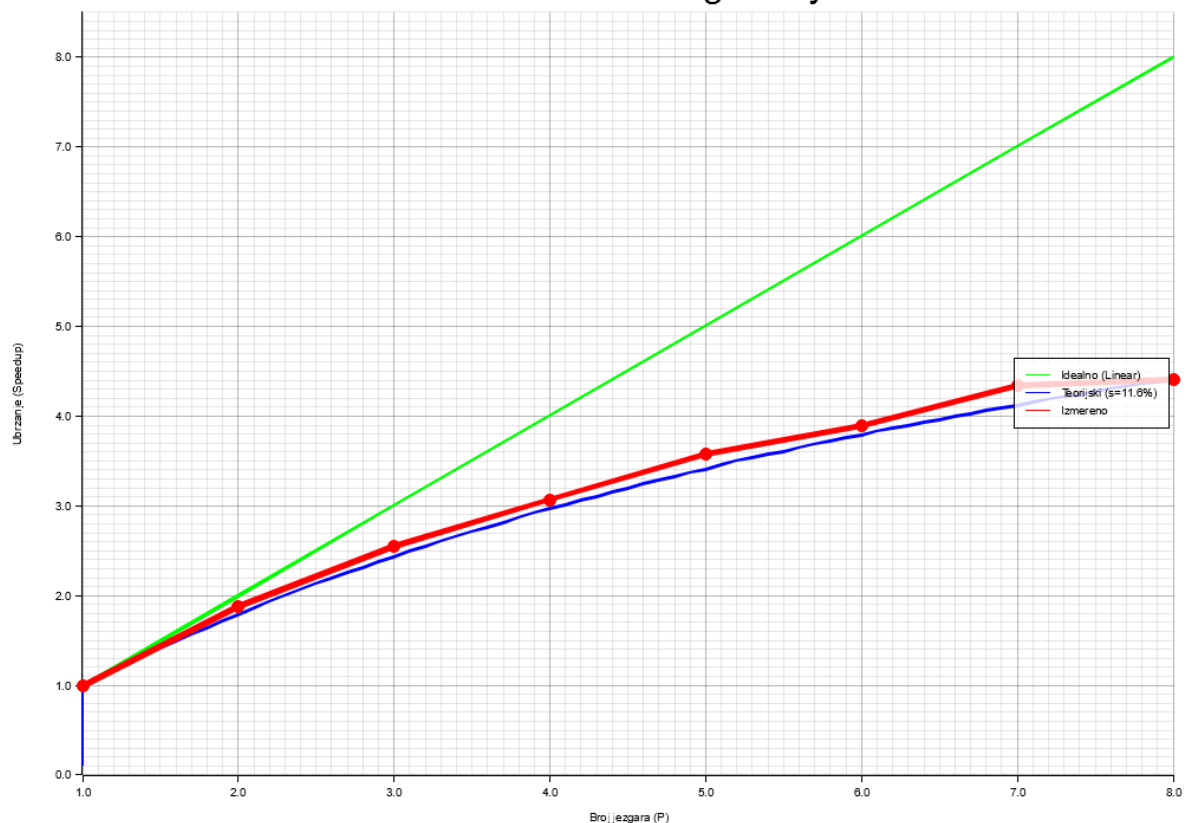
Prema Amdalovom zakonu, čak i da uposlimo beskonačan broj jezgara ($N \rightarrow \infty$), maksimalno teoretsko ubrzanje za ovaj algoritam bi bilo ograničeno na:

$$S_{\max} = 1/s$$

Gde malo s predstavlja procenat sekvencijanog dela koji ne može biti paralelizovan tako da je u mom slučaju maksimalno moguće ubrzanje za Python implementaciju približno 4.12 dok je kod Rust implementacije 8.62 što je više nego duplo u poređenju sa implementacijom rešenja u programskom jeziku Python.



RUST STRONG Scaling Analysis



Slabo skaliranje (Gustafsonov zakon)

Teorija

Za razliku od Amdalovog zakona, koji posmatra fiksnu veličinu problema, Gustafsonov zakon pretpostavlja da se veličina problema povećava proporcionalno broju procesora.

Formula: $S(N) = s + p \cdot N$

- N je broj procesora
- s je proporcija vremena izvršavanja provedena na serijskom delu koda
- p je proporcija vremena izvršavanja provedena na delu koda koji se može paralelizovati

U idealnom slučaju, vreme izvršavanja treba da bude konstantno.

Eksperiment

U konkretnom eksperimentu, broj masina se povećavao po formuli $10 \cdot \text{broj_jezgara}$ dok je broj simulacija po mašini bio 10000 i broj godina za simulaciju 30. Broj simulacija po jezgru je 100000.

U nastavku su prikazani tabelarni podaci nakon testiranja paralelne implementacije rešenja korišćenjem Python i Rust programskih jezika.

Podaci za Python:

Broj jezgara (N)	Broj simulacija	Mean Time (s)	Std Dev	Outliers (IQR)
1	100000	7.9330	3.934	2
2	200000	9.4336	6.536	0
3	300000	11.2028	5.066	3
4	400000	13.1064	5.619	2
5	500000	14.6478	4.740	3
6	600000	15.2200	2.789	6
7	700000	16.3063	2.575	0
8	800000	18.0937	2.681	3

Podaci za Rust:

Broj jezgara (N)	Broj simulacija	Mean Time (s)	Std Dev	Outliers (IQR)
1	100000	0.3265	0.0056	4
2	200000	0.3791	0.0203	1
3	300000	0.4253	0.0169	1
4	400000	0.4179	0.0121	2
5	500000	0.4605	0.0290	1
6	600000	0.4718	0.0160	4
7	700000	0.5262	0.0455	0
8	800000	0.5736	0.0385	0

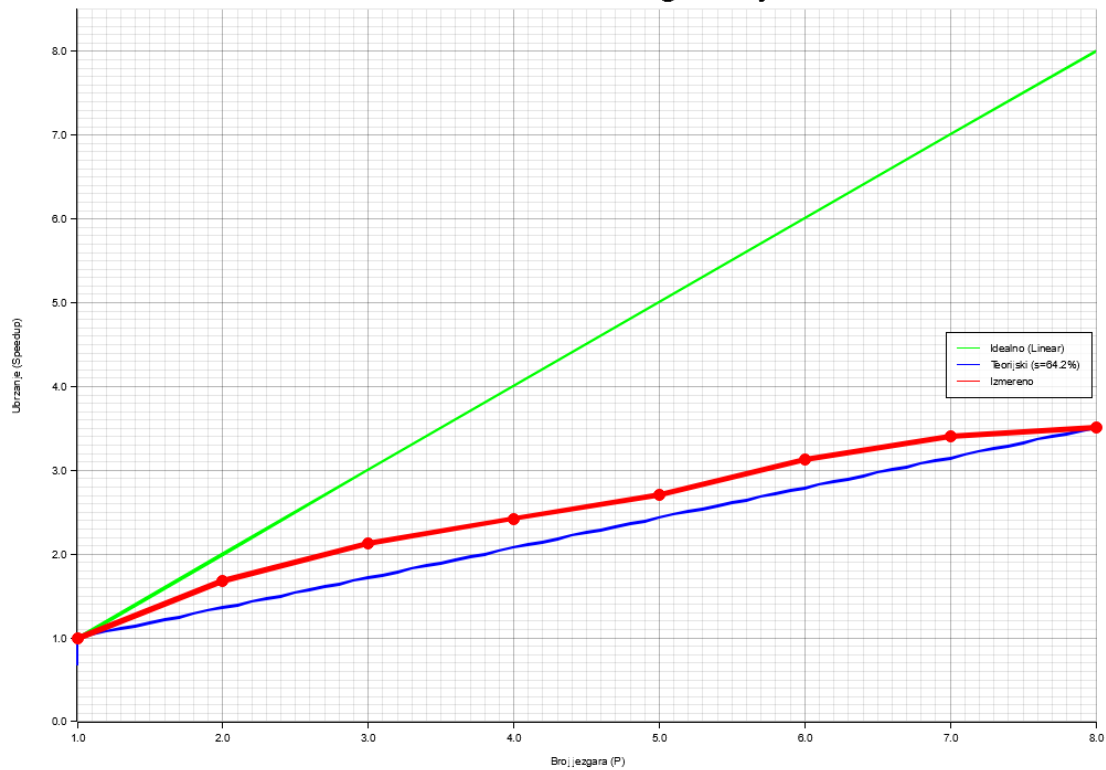
Analiza eksperimenta

Kao što se i moglo pretpostaviti, za isti obim posla implementacija u Rust programskom jeziku postiže značajno bolje rezultate. Vreme potrebno da se uradi proporcionalna ista količina posla za 1 i 8 jezgara procesa je porasla za 0.2 sekunde. U implementaciji koja koristi programski jezik Python vreme je poraslo za skoro 10.2 sekunde.

Stvarno (izmereno) ubrzanje $S(N)$: Računa se kao odnos rada obavljenog na N jezgara u odnosu na rad na jednom jezgru, normalizovan vremenom. Za slabo skaliranje, formula je: $S(N) = N \cdot T(1) / T(N)$. U mom slučaju, stvarno ubrzanje na 8 jezgara iznosi približno 3.51 za implementaciju pomoću Python programskog jezika i 4.55 za implementaciju pomoću Rust programskog jezika. Teorijsko ubrzanje iznosi približno 6.3 za Python i 7.19 za Rust.

Efikasnost je izračunata po formuli kao i za jako skaliranje i iznosi približno 43.8% kod Python implementacije i 56.9% kod Rust implementacije.

PYTHON WEAK Scaling Analysis



RUST WEAK Scaling Analysis

